

А. Агроном-любитель

Полное решение

Ограничение времени 2 секунды

Ограничение памяти 64 Мб

Ввод стандартный ввод или agro.in

Вывод стандартный вывод или agro.out

Бородавской школьник Лёша поехал на лето в деревню и занялся выращиванием цветов. Он посадил n цветков вдоль одной длинной прямой грядки, и они успешно выросли. Лёша посадил множество различных видов цветков, i -й от начала грядки цветок имеет вид a_i , где a_i — целое число, номер соответствующего вида в «Каталоге юного агронома».

Теперь Лёша хочет сделать фотографию выращенных им цветов и выложить ее в раздел «мои грядки» в социальной сети для агрономов «ВКомпосте». На фотографии будет виден отрезок из одного или нескольких высаженных подряд цветков.

Однако он заметил, что фотография смотрится не очень интересно, если на ней много одинаковых цветков подряд. Лёша решил, что если на фотографии будут видны три цветка одного вида, высаженные подряд, то его друзья — специалисты по эстетике цветочных фотографий — поставят мало лайков.

Помогите ему выбрать для фотографирования как можно более длинный участок своей грядки, на котором нет трех цветков одного вида подряд.

Формат ввода

В первой строке содержится целое число n ($1 \leq n \leq 200000$) — количество цветков на грядке.

Во второй строке содержится n целых чисел a_i ($1 \leq a_i \leq 109$), обозначающих вид очередного цветка. Одинаковые цветки обозначаются одинаковыми числами, разные — разными.

а

Формат вывода

Выведите номер первого и последнего цветка на самом длинном искомом участке. Цветки нумеруются от 1 до n .

0

Если самых длинных участков несколько, выведите участок, который начинается раньше.

0

Пример

Ввод	Вывод
6 5 6 6 6 23 9	3 6

```

#include <iostream>
#include <vector>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    int left = 0;
    int max_len = 0;
    int max_left = 0;
    int max_right = 0;
    for (int right = 0; right < n; right++) {
        while (right >= 2 && a[right] == a[right - 1] && a[right - 1] == a[right - 2]) {
            left = right - 1;
            break;
        }

        int current = right - left + 1;
        if (current > max_len) {
            max_len = current;
            max_left = left;
            max_right = right;
        }
    }

    cout << max_left + 1 << " " << max_right + 1 << endl;

    return 0;
}

```

Объяснение:

Суть задачи состоит в нахождении самого длинного непрерывного участка, на котором нет трех цветков одного вида подряд. Для решения этой задачи я использую метод двух указателей. `left` - левая граница текущего допустимого участка, `right` - правая граница. Основная идея решения состоит в том, что мы расширяем участок вправо (`right ++`) и, если находим три одинаковых цветка подряд, сдвигаем `left` так, чтобы нарушающий элемент остался за пределами участка (оставляем только два последних цветка). Алгоритм работает за $O(n)$ – один проход по массиву, по общей памяти за $O(n)$ – хранение массива.

В. Зоопарк Глеба

Полное решение

Ограничение времени 1 секунда

Ограничение памяти 256 Мб

Ввод стандартный ввод или input.txt

Вывод стандартный вывод или output.txt

Недавно Глеб открыл зоопарк. Он решил построить его в форме круга и, естественно, обнёс забором. Глеб взял вас туда начальником охраны. Казалось бы все началось так хорошо, но именно в вашу первую смену все животные разбежались. В зоопарке n животных различных видов, также под каждый из видов есть свои ловушки. К сожалению некоторые животные враждуют с друг другом в природе (они обозначены разными буквами), а зоопарк обнесён забором и имеет форму круга. С помощью камер, удалось выяснить, где находятся все животные. Умная система поддержки жизнедеятельности зоопарка уже просканировала зоопарк и вывела id всех животных и ловушек в том порядке, в котором они видны из центра зоопарка. Получилось так, что все животные и все ловушки находятся на краю зоопарка. Вы хотите понять, могут ли животные прийти в свою ловушку так, чтобы их путь не пересекался с другими. Если да, также предъявите какую-нибудь из схем поимки животных.

Формат ввода

На вход подается строка из $2 \cdot n$ символов латинского алфавита, где маленькая буква - животное, а большая - ловушка. Размер строки не более 100000.

0

Формат вывода

Требуется вывести "Impossible", если решения не существует или "Possible", если можно вернуть всех животных в клетки. В случае если можно, то для каждой ловушки в порядке обхода требуется вывести индекс животного в ней.

Пример 1

Ввод	Вывод
ABba	Possible
	2 1

Пример 2

Ввод	Вывод
ABab	Impossible

```

#include <cctype>
#include <iostream>
#include <stack>
#include <string>
#include <vector>

using namespace std;

void solve() {
    string s;
    if (!(cin >> s))
        return;

    int n = s.length();

    vector<int> animal_id(n, 0);
    int c = 0;

    for (int i = 0; i < n; ++i) {
        if (islower(s[i])) {
            c++;
            animal_id[i] = c;
        }
    }

    stack<int> st;
    vector<int> id_animal(n, 0);

    for (int i = 0; i < n; ++i) {
        if (!st.empty()) {
            int ind = st.top();
            if (tolower(s[i]) == tolower(s[ind]) && s[i] != s[ind]) {
                int lov = -1;
                int animal_ind = -1;

                if (isupper(s[i])) {
                    lov = i;
                    animal_ind = ind;
                } else {
                    lov = ind;
                    animal_ind = i;
                }

                id_animal[lov] = animal_id[animal_ind];

                st.pop();
                continue;
            }
        }

        st.push(i);
    }
}

```

```

if (!st.empty()) {
    cout << "Impossible" << endl;
} else {
    cout << "Possible" << endl;
    for (int i = 0; i < n; ++i) {
        if (isupper(s[i])) {
            cout << id_animal[i] << " ";
        }
    }
    cout << endl;
}
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    solve();
    return 0;
}

```

Объяснение:

Суть задачи состоит в определении, могут ли все животные добраться до своих ловушек так, чтобы их пути не пересекались. Если такое возможно, то нужно, помимо сообщения, вывести номер животного для каждой ловушки. Я заметил, что эта задача похожа на задачу о проверке правильной скобочной последовательности с несколькими типами скобок. Для решения этой задачи я использовал стек, в который добавлял животное или ловушку когда встречал их и если у него уже не находилось пары в верхушке стека (если нет пары в верху стека — кладем в стек, если есть пара — удаляем из стека). Если же встречаем пару в верхушке стека, то удаляем из стека. В конце, если стек оказался пустым, то все пары найдены и животные смогли добраться до ловушек так, чтобы их пути не пересекались. Алгоритм работает за $O(n)$ — один проход по строке, по общей памяти за $O(n)$ (в стеке может храниться до n элементов).

С. Конфигурационный файл

Полное решение

Ограничение времени 1 секунда

Ограничение памяти 64 Мб

Ввод стандартный ввод или input.txt

Вывод стандартный вывод или output.txt

Вадим разрабатывает парсер конфигурационных файлов для своего проекта. Файл состоит из блоков, которые выделяются с помощью символов «{» — начало блока, и «}» — конец блока. Блоки могут вкладываться друг в друга. В один блок может быть вложено несколько других блоков.

В конфигурационном файле встречаются переменные. Каждая переменная имеет имя, которое состоит из не более чем десяти строчных букв латинского алфавита. Переменным можно присваивать числовые значения. Изначально все переменные имеют значение 0.

Присваивание нового значения записывается как `<variable>=<number>`,

где `<variable>` — имя переменной, а `<number>` — целое число, по модулю не превосходящее 109. Парсер читает конфигурационный файл построчно. Как только он встречает выражение присваивания, он присваивает новое значение переменной. Это значение сохраняется до конца текущего блока, а затем восстанавливается старое значение переменной. Если в блок вложены другие блоки, то внутри тех из них, которые идут после присваивания, значение переменной также будет новым.

Кроме того, в конфигурационном файле можно присваивать переменной значение другой переменной. Это действие записывается как `<variable1>=<variable2>`. Прочитав такую строку, парсер присваивает текущее значение переменной `variable2` переменной `variable1`. Как и в случае присваивания константного значения, новое значение сохраняется только до конца текущего блока. После окончания блока переменной возвращается значение, которое было перед началом блока.

Для отладки Вадим хочет напечатать присваиваемое значение для каждой строки вида `<variable1>=<variable2>`. Помогите ему отладить парсер.

Формат ввода

Входные данные содержат хотя бы одну и не более 105 строк. Каждая строка имеет один из четырех типов:

- { — начало блока;
- } — конец блока;
- `<variable>=<number>` — присваивание переменной значения, заданного числом;
- `<variable1>=<variable2>` — присваивание одной переменной значения другой переменной. Переменные `<variable1>` и `<variable2>` могут совпадать.

Гарантируется, что ввод является корректным и соответствует описанию из условия. Ввод не содержит пробелов.

Формат вывода

Для каждой строки типа `<variable1>=<variable2>` выведите значение, которое было присвоено.

Пример

Ввод	Вывод
a=b	
b=123	
var=b	
b=-34	0
{	123
c=b	-34
b=10000000000	10000000000
d=b	10000000000
{	123
a=b	-34
e=var	
}	
}	
b=b	

```
#include <cctype>
#include <iostream>
#include <map>
#include <stack>
#include <string>
#include <vector>
```

```
using namespace std;
```

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
```

```
    map<string, long long> variables;
    stack<vector<pair<string, long long>>> history;
```

```
    string s;
    while (cin >> s) {
        if (s == "{") {
            history.push({});
        } else if (s == "}") {
            for (auto it = history.top().rbegin(); it != history.top().rend(); ++it) {
                variables[it->first] = it->second;
            }
            history.pop();
        } else {
            size_t pos = s.find('=');
            string var1 = s.substr(0, pos);
            string right = s.substr(pos + 1);
            if (isdigit(right[0]) || right[0] == '-') {
                long long num = stoll(right);
                if (!history.empty()) {
                    history.top().push_back({var1, variables[var1]});
                }
            }
        }
    }
}
```

```

    variables[var1] = num;
} else {
    cout << variables[right] << "\n";
    if (!history.empty()) {
        history.top().push_back({var1, variables[var1]});
    }
    variables[var1] = variables[right];
}
}
}
return 0;
}

```

Объяснение:

В задаче есть блоки (`{` - начало блока, `}` - конец блока). Я сохраняю значение переменной только до конца текущего блока и, после выхода из блока, восстанавливаю предыдущее значение. Для этого я использую `map` для хранения текущих значений и `stack` для хранения только измененных переменных в блоке. При восстановлении я использую обратный порядок — это гарантирует восстановление исходного значения, если переменная меняется несколько раз в одном блоке. Алгоритм работает за $O(n \cdot l)$, n строк и l — длина имени переменной, по общей памяти за $O(n \cdot l)$ — стек + `map` переменных.

Д. Профессор Хаос

Полное решение

Ограничение времени 1 секунда

Ограничение памяти 64 Мб

Ввод стандартный ввод или chaos.in

Вывод стандартный вывод или chaos.out

В секретной лаборатории профессора Хаоса проходит эксперимент по выращиванию особо опасных бактерий. В начале первого дня эксперимента у Хаоса имеется a особо опасных бактерий.

Каждый день эксперимента устроен следующим образом. Рано утром профессор достает из контейнера все свои бактерии и помещает их в инкубатор, где бактерии начинают делиться. Вместо каждой бактерии образуется b новых бактерий.

После извлечения бактерий из инкубатора c из них используются для проведения различных опытов и затем уничтожаются. Если после извлечения из инкубатора имеется менее c бактерий, для проведения опытов используются все имеющиеся бактерии, и эксперимент заканчивается.

Оставшиеся бактерии в конце дня необходимо поместить в контейнер и продолжить использовать в эксперименте. Однако в контейнер можно поместить не более d бактерий, поэтому если число оставшихся бактерий больше d , то в контейнер помещаются d бактерий, а остальные уничтожаются.

Теперь профессор Хаос хочет выяснить, сколько особо опасных бактерий будет у него в контейнере после k -го дня эксперимента. Помогите ему найти ответ на этот вопрос.

Формат ввода

В единственной строке входного файла содержится пять целых чисел a, b, c, d и k ($1 \leq a, b \leq 1000, 0 \leq c \leq 1000, 1 \leq d \leq 1000, a \leq d, 1 \leq k \leq 1018$).

И

Формат вывода

Выведите одно число — количество бактерий у Хаоса к концу k -го дня. Если эксперимент завершится в k -й день или ранее, выведите число 0.

И

Пример 1

	Ввод	Вывод
0	3 1 5 2 5	

Пример 2

	Ввод	Вывод
1	2 0 4 3 4	

Пример 3

	Ввод	Вывод
1	2 3 5 2 0	

```

#include <cctype>
#include <iostream>
#include <map>
#include <string>
#include <vector>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    vector<long long> s(5);
    for (int i = 0; i < 5; ++i) {
        cin >> s[i];
    }

    long long a = s[0];
    long long b = s[1];
    long long c = s[2];
    long long d = s[3];
    long long k = s[4];
    map<long long, long long> history;
    long long day = 0;
    while (day < k) {
        if (history.find(a) != history.end()) {
            long long cycle_start = history[a];
            long long cycle_len = day - cycle_start;
            long long end = k - day;

            day += (end / cycle_len) * cycle_len;
            if (day >= k) {
                break;
            }
            history.clear();
        }

        history[a] = day;
        a = a * b;
        if (a < c) {
            cout << 0 << endl;
            return 0;
        }

        a -= c;
        if (a > d) {
            a = d;
        }

        day++;
    }

```

```
}  
cout << a << endl;  
return 0;  
}
```

Объяснение:

Задача требует найти количество бактерий после k дней, где k может быть до 10^{18} . Простая симуляция не пройдет по времени (не проходится 15 тест). Я заметил, что значение a ограничено сверху числом d (≤ 1000). Это значит, что a может принимать только 1000 различных значений, то есть рано или поздно значение a повторится $\rightarrow$ начнется цикл. Для определения цикла я храню в `map` историю состояний ($a \rightarrow$ день). Если значение a повторяется, то вычисляем длину цикла, а затем пропускаем полные циклы и продолжаем симуляцию до конца. Алгоритм работает за $O(\min(k, d))$, по памяти за $O(d)$ – `map` хранит до d состояний.